



SAPIENZA
UNIVERSITÀ DI ROMA

DEPARTMENT OF COMPUTER SCIENCE

From Models to Suites: FSM-Based Robot Test Generator

ADVANCED SOFTWARE ENGINEERING

Professors:

Paolo Bottoni

Students:

Lucian Dorin Crainic

1938430

Academic Year 2024/2025

Abstract

This document presents the research conducted on FSM-based test generation for robotic systems. The focus is on the methodologies employed, the results obtained, and the implications for future research in automated testing.

Contents

1	Introduction	3
1.1	Motivation: Why FSM-based Testing?	3
1.2	Project objective and scope	3
1.3	Main contributions	3
2	Background and Related Work	5
2.1	Finite-State Machines (FSMs) for Test Generation	5
2.2	Robot Framework Essentials	6
2.3	Coverage Criteria (State, Transition, Action)	7
2.4	Comparable Tools and Approaches	7
3	FSM Grammar Specification (.machine)	9
3.1	Design Goals	9
3.2	Complete EBNF Grammar	10
3.3	Semantic Validation Rules	11
4	Generator Design and Implementation	14
4.1	High-Level Design	14
4.2	Main data flow	14
4.3	Strategy Interface and Implementations	14
4.4	Core Implementation highlights	14
5	Verification and Validation	15
5.1	Unit Tests	15
5.2	Integration Tests	15
5.3	Login FSM example	15
6	Real-project case study	16
6.1	System description	16
6.2	Typical workflow in practice	16
6.3	Different test generation results	16
6.4	Discussion and future work	16
	References	17
	References	17

1 Introduction

1.1 Motivation: Why FSM-based Testing?

Finite-state machines (FSMs) offer a concise way to model application behaviour. Each state captures a distinct configuration, while actions encode the events that drive the system. By representing a workflow as an FSM we can automatically explore and verify the possible transitions. Automated test generation from models reduces the effort spent writing exhaustive suites by hand and helps highlight unreachable or missing scenarios.

1.2 Project objective and scope

This project provides a prototype test generator that transforms `.machine` models into Robot Framework test suites. The generator is implemented in Python and is organised around a parsing layer, a set of state-based data structures and several generation strategies. The main modules include:

- **parsing.py** — reads the custom grammar and builds objects such as `Machine`, `State` and `Action`.
- **generator.py** — walks the model and emits test cases, keeping track of coverage information.
- **strategies.py** — contains depth-first, random and pairwise algorithms for exploring the state space.

To demonstrate the generated tests, a simple shopping web application is provided. A `FastAPI` backend (see `backend/main.py`) exposes endpoints for login, product management and checkout, while a React frontend uses the API to implement an FSM driven user interface.

1.3 Main contributions

The repository delivers a minimal yet complete pipeline from model to executable tests. Key features are:

- a human-readable `.machine` grammar for defining states, actions, variables and rules;
- a parser capable of validating expressions and resolving variable references;
- generation strategies that produce Robot Framework suites with optional coverage comments;

- an example web application with unit tests to showcase integration with real code.

Figure 1: Overall architecture of the prototype generator.

2 Background and Related Work

2.1 Finite-State Machines (FSMs) for Test Generation

Finite-state machines (FSMs) describe software behavior as a collection of states and the transitions that connect them. Each transition is triggered by an input and may produce an output, making FSMs well suited for modelling reactive systems[1]. Test generation from an FSM typically begins with a formal specification that enumerates all possible states of the system under consideration. From this specification, automated tools derive sequences of inputs that traverse the state space. The resulting test cases explore systematic paths through the model and help uncover defects that manifest only after a particular series of events.

Early research established a direct link between FSM models and systematic test coverage. Chow[2] introduced the W-method to generate tests that exercise every transition of a minimal deterministic machine. Subsequent work extended these concepts to non-deterministic and partially specified models. A common pattern among these methods is to treat the FSM as a graph and search for paths that satisfy specific coverage goals. For instance, breadth-first search can be used to create a test suite that visits every state at least once, while more advanced techniques may incorporate heuristics for minimising the number of steps.

Figure 2 illustrates a small FSM representing a user login workflow. The model identifies valid credentials, incorrect attempts, and the lockout state after successive failures. Although the diagram is simplistic, it highlights how model-based tests can capture user interactions with strict control over the order of events. Complex systems typically contain many more states, and exhaustive enumeration quickly becomes impractical. Therefore, researchers have proposed abstraction and compositional reasoning to keep the models tractable. Such approaches allow testers to focus on critical regions of the state space without enumerating every permutation.

Figure 2: Example finite-state machine representing a simplified login process.

In modern development workflows, FSM-based test generation forms the backbone of many model-based testing (MBT) tools. These tools integrate with continuous integration pipelines to execute automatically derived tests whenever the model or system under test changes. Despite progress in automation, practitioners must still craft models carefully to balance completeness and maintainability. The remainder of this chapter discusses how FSMs integrate with the Robot Framework and how various coverage criteria ensure the generated tests provide meaningful assurance.

2.2 Robot Framework Essentials

The Robot Framework is an open-source automation platform designed for acceptance and end-to-end testing. Tests in Robot Framework are written in a tabular syntax, which emphasises readability and encourages collaboration between developers and nontechnical stakeholders[3]. At the core of Robot Framework is a keyword-driven approach, where keywords represent high-level actions or verifications. Built-in and external libraries provide keywords for interacting with web browsers, databases, application interfaces, and more.

Robot Framework test suites follow a consistent structure consisting of settings, variables, test cases, and optional keywords. Settings declare resources such as libraries, resource files, and metadata. Variables simplify parameterisation by making values reusable across the suite. The test cases themselves describe a series of keywords executed in order. By defining custom keywords, teams can encapsulate complex actions and create a domain-specific language tailored to their application.

Listing 1 shows a minimal Robot Framework test that verifies a successful login. The script opens a browser, enters credentials, and checks for a welcoming element on the resulting page. The example uses the SeleniumLibrary, one of the most widely adopted libraries for web testing. While simple, the listing demonstrates how a few lines in Robot syntax orchestrate a complete test scenario.

```
1 *** Settings ***
2 Library           SeleniumLibrary
3
4 *** Test Cases ***
5 Valid Login
6     Open Browser    ${LOGIN_URL}    chrome
7     Input Text      id=username      demo
8     Input Text      id=password      secret
9     Click Button     id=login
10    Page Should Contain Element    css=.welcome
11    [Teardown]        Close Browser
```

Listing 1: Simple login test in Robot Framework

Robot Framework supports a rich ecosystem of plugins and tools. Tags can be used for selective execution, and listeners provide hooks for custom reporting. The framework integrates with continuous integration servers and allows tests to be written in plain text, reStructuredText, or Markdown formats. Because of its extensibility, Robot Framework is an ideal host environment for model-based test generation, where models can be translated into high-level keywords that map directly to system interactions.

2.3 Coverage Criteria (State, Transition, Action)

Coverage metrics help quantify how thoroughly a set of test cases exercises the model of a system. For FSM-based approaches, three fundamental criteria are commonly considered: state coverage, transition coverage, and action coverage. State coverage requires every reachable state in the model to appear in at least one test case. Achieving full state coverage ensures the test suite observes the system in all of its documented configurations, though it does not guarantee that every change of state is exercised.

Transition coverage goes a step further by insisting that each directed edge in the FSM be executed by some test. This criterion verifies that the system under investigation handles all allowable inputs at every decision point. Various algorithms exist to derive a minimal set of paths that collectively cover every transition[4]. In practice, complete transition coverage may be unreachable for complex or nondeterministic systems, so engineers often focus on critical subsets of transitions.

Action coverage pertains to the operations or outputs associated with transitions. An action might correspond to a method invocation, a message sent on a queue, or an update to persistent state. Action coverage requires that each unique action defined in the FSM is observed during execution. This criterion is useful when transitions share common labels or when additional side effects need verification beyond state changes. Some tools track action coverage by instrumenting the test harness or by parsing logs to associate actions with model elements.

Existing versions of this project provide basic state and transition tracking, but finer-grained action coverage is earmarked for future work. Integrating coverage tooling directly into the Robot Framework environment will allow continuous measurement as tests execute, providing immediate feedback on which parts of the model remain untested.

2.4 Comparable Tools and Approaches

Multiple model-based testing tools employ FSMs or related formalisms to generate test cases. GraphWalker is an open-source tool that interprets models described in a graph notation and executes random or guided walks to produce test sequences[5]. It integrates with Java and allows assertions to be placed on edges or vertices. Modbat is another approach, offering a Scala-based environment for modelling probabilistic automata[6]. Modbat can express complex concurrency scenarios and uses randomised exploration combined with user-defined assertions.

Tcases focuses on input space modelling rather than explicit state machines. It employs combinatorial techniques to derive minimal test sets that cover interactions among input parameters[7]. Although Tcases does not rely on an FSM, it shares the goal of systematic coverage and can complement state-based approaches. Other tools, such as QuickCheck and its derivatives, utilise property-based testing to generate large

numbers of input sequences that satisfy high-level properties.

Tool	Model Type	Notable Feature
GraphWalker	Graph-based FSM	Guided and random traversals
Modbat	Probabilistic automata	Concurrency support
Tcases	Input parameter model	Combinatorial coverage
QuickCheck	Property specifications	Randomised test data

Table 1: Representative tools for model-based test generation.

Compared with these tools, the present project emphasises ease of integration with the Robot Framework and straightforward model specification. While GraphWalker and Modbat target Java or Scala environments, our approach focuses on Python-based tooling and the readable syntax favoured by Robot Framework users. Tcases and QuickCheck address different kinds of models, yet they share complementary goals of systematic exploration. The methodology outlined in this work seeks to combine the accessibility of Robot Framework with the thoroughness of MBT techniques. The following chapter introduces the concrete design and implementation choices made to realise this integration.

3 FSM Grammar Specification (.machine)

3.1 Design Goals

The “.machine“ format defines finite state machines used by the generator in `tests/generator`. Its syntax resembles Robot Framework tables so that engineers accustomed to Robot’s plain text style can author models with minimal training. The grammar is intentionally concise: every section begins with a triple-asterisk header (for example “*** Machine ***”) and relies on consistent indentation rather than explicit block markers. This yields readable models and ensures that hand-crafted machines can easily be versioned alongside tests.

A primary goal is to separate variable declarations from state transitions. The **Variables** section enumerates parameter values that may vary across test cases. During generation, the engine combines these values with the state model to produce different execution paths. Rules placed after the variable block constrain valid combinations, preventing unrealistic or contradictory settings.

Within the **Machine** section each state appears at column 0 and may include optional step lines describing actions to execute upon entering the state. An **[Actions]** header introduces transitions triggered by keywords. Each action specifies the target state and can include a conditional expression using the keyword “when”. Actions with the label “otherwise” act as fallbacks when no other condition matches. Arguments may be listed using the “args” keyword so that values are resolved when tests are generated.

The format also supports optional **Settings** and **Keywords** sections. Content placed in the **Settings** area is copied verbatim to the resulting Robot Framework file, allowing resource imports or test setup steps. The **Keywords** section provides reusable keyword implementations or helper functions referenced from the state machine. Because the parser stores the entire text of these sections, users retain complete control over the generated script without modifying the underlying generator.

Overall, the design emphasises human readability while remaining expressive enough for automated processing. By mirroring Robot Framework conventions and limiting syntactic noise, the grammar serves as a compact bridge between FSM models and executable tests. Another goal is to keep the syntax simple enough for domain experts who may not be familiar with Python. Because every statement is plain ASCII, machine files can be edited with standard text editors and reviewed like source code. The format intentionally avoids complex nesting or macros. Instead, extensibility is achieved through keywords that encapsulate reusable sequences of Robot commands. By isolating variable declarations, state descriptions, and reusable keyword implementations, the grammar encourages small, composable models that evolve with the system under test.

In practice, the parser supports both a lightweight and a strict mode. The `parse_simple`

function reads each line sequentially and tolerates minor formatting variations. If this fast path fails, the more rigorous pyparsing implementation reports the exact line and column of the offending construct. This dual approach balances flexibility for authors with precise diagnostics when errors occur.

3.2 Complete EBNF Grammar

The following listing summarises the entire grammar recognised by the parser in `tests/generator/src/machine/parsing.py`. Long rules are wrapped for clarity.

```

1 machine_file      = settings_section? variables_section? machine_section
2                   keywords_section? ;
3 settings_section = "*** Settings ***" newline text* ;
4 variables_section= "*** Variables ***" newline variable_def* ;
5 variable_def     = variable splitter "any of" splitter value_list newline ;
6 variable         = "${" identifier "}" ;
7 value_list       = value ( splitter value )* ;
8 value            = token ;
9 machine_section  = "*** Machine ***" newline elements ;
10 elements        = ( variable_def | rule | state_def )* ;
11 rule            = expression splitter "==" splitter expression newline ;
12 state_def       = state_name newline step* actions? ;
13 state_name      = identifier (" " identifier)* ;
14 step            = two_spaces text newline ;
15 actions         = two_spaces "[Actions]" newline action+ ;
16 action          = four_spaces robo_step splitter "==" splitter state_name
17                  condition? args? newline ;
18 condition       = splitter "when" splitter expression
19                  | splitter "otherwise" ;
20 args            = splitter "args" splitter value_list ;
21 keywords_section= "*** Keywords ***" newline text* ;
22 expression      = term ( ("and" | "or") term )* ;
23 term            = variable comparison_op value
24                  | "(" expression ")" ;
25 comparison_op   = "==" | "!=" | ">" | "<" | ">=" | "<=" | "in" | "~" | "!~" ;
26 identifier      = letter ( letter | digit | "_" )* ;
27 text            = {any characters other than newline};
28 newline         = "\n" | "\r\n" ;
29 splitter        = " " + " " ;
30 two_spaces      = "  " ;
31 four_spaces     = "    " ;

```

Listing 2: Full .machine grammar

This grammar covers every construct accepted by the parser. A settings section collects arbitrary lines until the next header, enabling direct reuse of Robot Framework directives. Variables follow the pattern **variable** **any** of **value_list**, where spacing is normalised by the **splitter** rule so that additional spaces are ignored. The indentation-based **step** and **action** productions remove the need for explicit block delimiters: two leading spaces indicate a step executed on entry, while four spaces introduce a transition. Conditions support boolean expressions composed of equality, inequality and comparison operators, as well as regular-expression matches using `~` and `!~`.

Tokens such as **identifier**, **text**, and **value** rely on regular expressions defined in the Python implementation. They accept alphanumeric characters and typical punctuation. The grammar remains intentionally compact so that future operators or features can be added with minimal disruption. In practice, the `parse_simple` routine tolerates minor variation but ultimately normalises input back to this canonical form before generation.

Figure 3: Overview of sections and their relation within a `.machine` file.

3.3 Semantic Validation Rules

Parsing proceeds in two stages. The function `parse()` first attempts a lightweight line-by-line parser called `parse_simple`. Should that fail, it falls back to a detailed `pyparsing` specification. Both approaches produce a `Machine` object composed of `State`, `Action`, and `Variable` instances. The simple parser explicitly records the original text of the **Settings**, **Variables**, and **Keywords** sections so that they can be emitted unchanged in the generated test file.

Every `Action` validates that its target state exists when it is attached to a machine. The following excerpt shows the relevant check:

```

1 class Action(object):
2     def set_machine(self, machine):
3         self._machine = machine
4         if not self.next_state:
5             raise AssertionError(
6                 f"Invalid end state '{self._next_state_name}' "
7                 f"in action '{self.name}'!")

```

Listing 3: Action state resolution

If a transition refers to an undefined state, an exception halts parsing, giving immediate feedback about typos or missing states.

Variables may be referenced inside action arguments or conditions. The `Variable` class resolves nested references while guarding against recursive substitution:

```

1 class Variable(object):
2     REGEX = r"\${[_A-Z][_A-Z0-9]*}"
3     PATTERN = re.compile(REGEX)
4     def _resolve_value(self, value):
5         if hasattr(self, '_resolving') and self._resolving:
6             return value
7         self._resolving = True
8         try:
9             return self.PATTERN.sub(self._resolve_variable, value)
10        finally:
11            self._resolving = False

```

Listing 4: Variable resolution logic

Global rules restrict combinations of variable values. Each rule is evaluated via its `is_valid()` method, and the `Machine.rules_are_ok` function only accepts assignments satisfying all rules. During test generation, strategies iterate over value sets filtered by this function.

```

1 class Machine(object):
2     def rules_are_ok(self, values):
3         value_mapping = {v.name: value for v, value in zip(self.variables, values)}
4         for rule in self.rules:
5             if not rule.is_valid(value_mapping=value_mapping):
6                 return False
7         return True

```

Listing 5: Rule evaluation pipeline

Indentation also carries semantic meaning. States must start at column 0, actions at four spaces, and steps at two spaces. The parser warns about tab characters and trims trailing whitespace to enforce uniform style. Lines beginning with “#” are treated as comments and ignored entirely.

The helper function `resolve_whitespace` prints a warning whenever a tab character is detected in the input file. This nudge helps maintain portability across editors that may mix tabs and spaces. After parsing succeeds, variable values are applied only if every rule evaluates to true. The generator also records which states and actions are executed while producing tests. This coverage data is reported as comments at the top of the generated file so that users can quickly see whether any portion of the model remained unexplored.

The overall validation process is summarised in Figure 4. The parser builds the machine structure, checks state references and variable rules, and then hands the validated model to the generator, which emits Robot Framework tests.

Figure 4: Parsing and validation pipeline from text to executable tests.

4 Generator Design and Implementation

4.1 High-Level Design

4.2 Main data flow

4.3 Strategy Interface and Implementations

4.4 Core Implementation highlights

5 Verification and Validation

5.1 Unit Tests

5.2 Integration Tests

5.3 Login FSM example

6 Real-project case study

6.1 System description

6.2 Typical workflow in practice

6.3 Different test generation results

6.4 Discussion and future work

References

- [1] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 2001.
- [2] T. S. Chow. Testing software design modeled by finite-state machines. *IEEE Transactions on Software Engineering*, SE-4(3):178–187, 1978.
- [3] Robot Framework Foundation. Robot framework user guide. <https://robotframework.org/>. Accessed: 2025-07-13.
- [4] Robert V. Binder. *Testing Object-Oriented Systems: Models, Patterns, and Tools*. Addison-Wesley, 1999.
- [5] GraphWalker Project. Graphwalker. <https://graphwalker.github.io/>. Accessed: 2025-07-13.
- [6] Matthias Heizmann. Modbat. <https://github.com/codelabs-ch/modbat>. Accessed: 2025-07-13.
- [7] Kerry Cornutum. Tcases. <https://github.com/Cornutum/tcases>. Accessed: 2025-07-13.